SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO3-ASSIGNMENT 3

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: CSA0601

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

Q1. DECOMPOSITION AND ELIMINATION IN ORDERED DATA

Part A: Finding Maximum and Minimum Using Divide and Conquer

1. Introduction

Finding the maximum and minimum elements in an array is a fundamental problem in algorithm design. Given a collection of numbers, the objective is to determine the largest and smallest values without unnecessarily repeating comparisons.

A straightforward approach scans the array to find the maximum and then scans it again to find the minimum. Although this method is simple, it performs two separate sequences of comparisons.

Divide and Conquer provides an alternative approach. Instead of processing the entire array as a single problem, it divides the array into smaller subproblems, solves them recursively, and combines their results to obtain the final maximum and minimum.

The main objective is to reduce the total number of comparisons while maintaining correctness.

2. Problem Definition

Given an array A containing n elements, find:

- The maximum element, which is the largest value in the array.
- The minimum element, which is the smallest value in the array.

For example, consider the array:

$$A = [12,5,19,3,27,8,14,2]$$

The maximum element is 27 , and the minimum element is 2.

The algorithm should return both values in a single solution.

Input: An array containing n elements, where $n \geq 1$

Output: The maximum and minimum elements of the array.

3. Limitations of the Independent Method

The independent method finds the maximum and minimum separately.

First, the algorithm initializes the maximum to the first element and compares every remaining element against it. It then initializes the minimum to the first element and performs another sequence of comparisons.

For n elements:

- Finding the maximum requires n-1 comparisons.
- Finding the minimum requires n-1 comparisons.

Therefore, the total number of comparisons is

$$C_{\text{independent}} = (n-1) + (n-1)$$

$$\boxed{C_{\text{independent}} = 2n - 2}$$

For an array of eight elements, this method requires

$$C_{\text{independent}} = 2(8) - 2 = 14$$

comparisons.

The method has linear time complexity, \(O(n)\), but performs more comparisons than necessary because maximum and minimum are determined independently.

4. Divide-and-Conquer Strategy

The Divide-and-Conquer method follows three fundamental stages.

- Divide: Split the array into two smaller parts.
- Conquer: Recursively determine the maximum and minimum of each part.
- Combine: Compare the maximum values returned by the two parts to obtain the overall maximum. Similarly, compare the two minimum values to obtain the overall minimum.

Suppose the array is divided into left and right halves.

Let the left half return:

$$(\max_L, \min_L)$$

and the right half return:

$$(\max_L, \min_L)$$

The combined maximum is

$$\max = \max(\max_L, \max_R)$$

The combined minimum is

$$\min = \min(\min_L, \min_R)$$

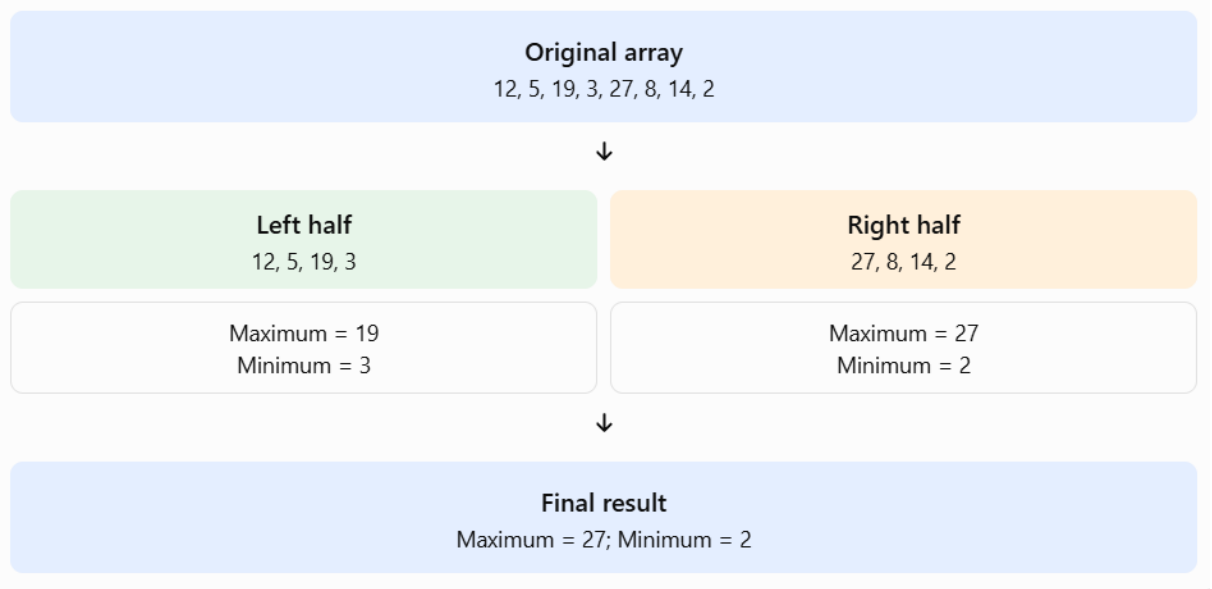
Only two comparisons are required to combine the results of two nontrivial subproblems.

5. Decomposition Representation

Consider the array:

$$[12,5,19,3,27,8,14,2]$$

The decomposition can be represented as follows:



The same process is applied recursively until each subproblem contains one or two elements.

6. Base Case

A recursive algorithm must have a base case to stop further division.

There are two useful base cases.

- Case 1: One element

If a subarray contains one element, that element is simultaneously its maximum and minimum.

$$\max = \min = A[l]$$

No comparison is necessary.

- Case 2: Two elements

If a subarray contains two elements, compare them once.

- If the first element is greater, it is the maximum and the second is the minimum.
- Otherwise, the second element is the maximum and the first is the minimum.

Only one comparison is required.

Using these base cases avoids unnecessary recursive calls and reduces the total comparison count.

7. Algorithm Pseudocode

Algorithm: FindMaxMin

Input: Array \(A\), lower index \(l\), upper index \(r\).

Output: Maximum and minimum values in the specified subarray.

ALGORITHM FindMaxMin(A, l, r)

INPUT:

A - array of elements

l - lower index

r - upper index

OUTPUT:

Maximum and minimum elements in A[l...r]

BEGIN

IF l = r THEN

RETURN (A[l], A[l])

END IF

IF r = l + 1 THEN

IF A[l] > A[r] THEN

RETURN (A[l], A[r])

ELSE

RETURN (A[r], A[l])

END IF

END IF

mid ← FLOOR((l + r) / 2)

(maxLeft, minLeft) ← FindMaxMin(A, l, mid)

(maxRight, minRight) ← FindMaxMin(A, mid + 1, r)

maximum ← MAX(maxLeft, maxRight)

minimum ← MIN(minLeft, minRight)

RETURN (maximum, minimum)

END

8. Python Implementation

```python
def find_max_min(arr, left, right):
    if left == right:
        return arr[left], arr[left]

    if right == left + 1:
        if arr[left] > arr[right]:
            return arr[left], arr[right]
        else:
            return arr[right], arr[left]

    mid = (left + right) // 2

    max_left, min_left = find_max_min(arr, left, mid)
    max_right, min_right = find_max_min(arr, mid + 1, right)

    maximum = max(max_left, max_right)
    minimum = min(min_left, min_right)

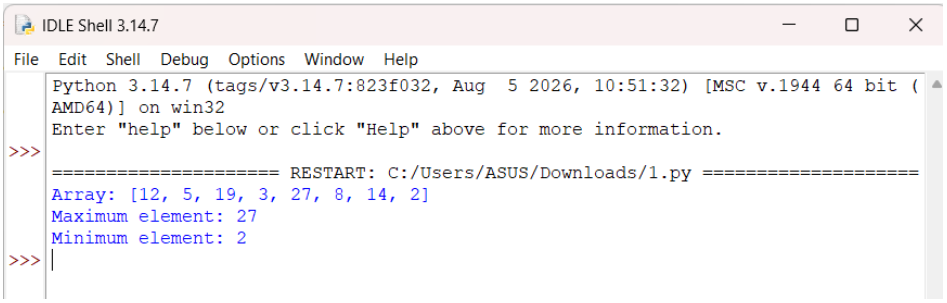
    return maximum, minimum


arr = [12, 5, 19, 3, 27, 8, 14, 2]

maximum, minimum = find_max_min(arr, 0, len(arr) - 1)

print("Array:", arr)
print("Maximum element:", maximum)
print("Minimum element:", minimum)
```

Output



9. Mathematical Analysis of Comparisons

Let $C(n)$ denote the number of comparisons required for an array of \(n\) elements.

For one element:

$$C(1) = 0$$

For two elements:

$$C(2) = 1$$

For larger arrays, the algorithm divides the input into two approximately equal subarrays and performs two additional comparisons during combination.

For a balanced split, the recurrence is

$$C(n) = C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + 2$$

For n equal to a power of two, this simplifies to

$$C(n) = 2C(n/2) + 2$$

Using the Master Theorem, the comparison count is linear. However, the recurrence can be solved more precisely.

For $n = 2^k$, the exact count is

$$\boxed{C(n) = \frac{3n}{2} - 2}$$

This expression applies to the implementation with one-element and two-element base cases and a balanced recursive split.

For $n = 8$:

$$C(8) = \frac{3(8)}{2} - 2 = 10$$

Thus, the Divide-and-Conquer algorithm requires 10 comparisons, compared with 14 for the independent method.

10. General Comparison Bound

For arbitrary n , the exact count depends on how many recursive subproblems contain odd numbers of elements and how the base cases are reached. The count remains linear.

A standard pairwise maximum-minimum method also achieves at most approximately

$$\left\lceil \frac{3n}{2} \right\rceil - 2$$

comparisons, with the precise integer expression depending on whether n is even or odd. The balanced recursive algorithm has the exact count given above when n is a power of two.

The important result is that the method reduces the number of comparisons while retaining

$$\left\lceil \frac{3n}{2} \right\rceil - 2$$

time complexity.

11. Comparison Table

Input size n	Independent method 2n-2	Divide and Conquer when n=2^k , frac{3n}{2}-2
2	2	1
4	6	4
8	14	10
16	30	22
32	62	46
64	126	94
128	254	190

The comparison savings become larger in absolute terms as the array grows.

For powers of two, the saving is

$$(2n - 2) - \left(\frac{3n}{2} - 2\right) = \frac{n}{2}$$

Thus, the method saves n/2 comparisons for these input sizes.

12. Interpretation of Experimental Results

The theoretical analysis predicts the number of element comparisons. The program provides an observed count that can be checked against the prediction.

If the observed count differs from the expected formula, the following factors should be examined:

- Whether the implementation uses the one-element and two-element base cases correctly.
- Whether comparisons in the combine stage are counted.
- Whether the program counts only element comparisons or also counts index and boundary checks.
- Whether the array size is a power of two.
- Whether the code performs additional comparisons during validation.

For example, a count larger than expected may indicate that a comparison has been counted twice or that a redundant comparison is being performed.

Execution time should be measured separately from comparison count. The comparison count is a deterministic measure of algorithmic work, whereas execution time can vary because of processor load,

interpreter overhead, memory behaviour, and other environmental factors.

13. Advantages and Limitations

Advantages

- Reduces the number of element comparisons compared with independently finding maximum and minimum.
- Uses a clear recursive structure.
- Demonstrates the Divide-and-Conquer design technique.
- Returns both results from the same traversal.
- Has linear time complexity.

Limitations

- Recursive function calls introduce overhead.
- The recursion consumes stack space.
- For very small arrays, an iterative pairwise implementation may be simpler.
- The implementation must correctly handle odd-sized subarrays and boundary conditions.

14. Conclusion

The Divide-and-Conquer strategy efficiently determines both the maximum and minimum elements by dividing the array into smaller parts and combining their results. Instead of performing two independent scans, the algorithm shares information obtained from the recursive subproblems.

For powers of two, the implementation requires 3n/2-2 element comparisons, compared with 2n-2 for the independent approach. Both methods have \(O(n)\) time complexity, but the Divide-and-Conquer method performs fewer comparisons.

The experiment demonstrates an important algorithm-design principle: an algorithm can improve its computational efficiency even when its asymptotic time complexity remains unchanged.

Part B: Binary Search Using Repeated Elimination

1. Introduction

Binary Search is an efficient searching algorithm used to locate a target value in a sorted array. Unlike Linear Search, which examines elements one by one, Binary Search repeatedly eliminates half of the remaining search space.

The main requirement is that the array must be sorted. The ordering provides enough information to determine whether the target can exist in the left half or the right half after each comparison.

For example, consider the sorted array:

$$A = [3,7,11,15,19,23,27,31,35]$$

Suppose the target is \(27\). Binary Search examines the middle element first. If the middle element is smaller than the target, every element to its left can be eliminated because the array is sorted.

The algorithm continues searching the remaining half until the target is found or no elements remain.

2. Problem Definition

Given a sorted array A containing n elements and a target value x , determine whether x is present in the array.

If the target exists, return its index. Otherwise, return NOT_FOUND.

Input:

- A sorted array A .
- The number of elements n .
- A target value x

Output: The index of the target, or NOT_FOUND.

3. Decision Rule

Binary Search maintains two boundaries:

- low: the first index of the current search interval.
- high: the last index of the current search interval.

The midpoint is calculated as

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

The algorithm compares the target with \(A[mid]\).

Case 1: Target found

If

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

the algorithm returns the midpoint.

- Case 2: Target is smaller

If

$$x < A[mid]$$

the target, if present, must lie to the left of the midpoint. Therefore,

$$high = mid - 1$$

- Case 3: Target is larger

If

$$x > A[mid]$$

the target, if present, must lie to the right of the midpoint. Therefore,

$$low = mid + 1$$

Each unsuccessful comparison eliminates the midpoint and approximately half of the remaining elements.

4. Why Is Elimination Correct?

The correctness of Binary Search depends on the sorted order of the array.

Suppose $A[mid] > x$. Since the array is sorted in ascending order, every element at an index greater than or equal to mid is at least $A[mid]$. Therefore, all those elements are greater than x , except that the midpoint itself has already been tested and found unequal.

Consequently, the target cannot occur in the right portion beginning at mid.

Similarly, if $A[mid] < x$, all elements at indices less than or equal to mid are no greater than $A[mid]$. Therefore, none of those elements can equal the larger target x.

This is why the eliminated portion does not need to be searched again.

The sorted-order condition is essential. Applying the same elimination rules to an unsorted array could discard the portion containing the target.

5. Worked Example: Successful Search

Consider:

$$A = [3,7,11,15,19,23,27,31,35]$$

Target:

$$x = 27$$

Step	Low	High	Mid	Middle value	Decision
1	0	8	4	19	$27 > 19$; eliminate left half
2	5	8	6	27	Target found

The target is located at index 6, which is the seventh position in one-based counting.

Only two midpoint comparisons are required.

6. Worked Example: Unsuccessful Search

Use the same array with target 26.

Step	Low	High	Mid	Middle value	Decision
1	0	8	4	19	Search right
2	5	8	6	27	Search left
3	5	5	5	23	Search right
4	6	5	—	—	Search interval is empty

The target is not found. When low > high, no candidate elements remain.

The algorithm terminates and returns NOT_FOUND.

7. Language-Independent Pseudocode

ALGORITHM BinarySearch(A, n, target)

INPUT:

A - sorted array of n elements

target - value to locate

OUTPUT:

Index of target, or NOT_FOUND

BEGIN

low ← 0

high ← n - 1

WHILE low ≤ high DO

mid ← FLOOR((low + high) / 2)

IF A[mid] = target THEN

RETURN mid

ELSE IF target < A[mid] THEN

high ← mid - 1

ELSE

low ← mid + 1

END IF

END WHILE

RETURN NOT_FOUND

END

8. Python Implementation With Comparison Counting

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2

        comparisons += 1

        if arr[mid] == target:
            return mid, comparisons

        comparisons += 1

        if target < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

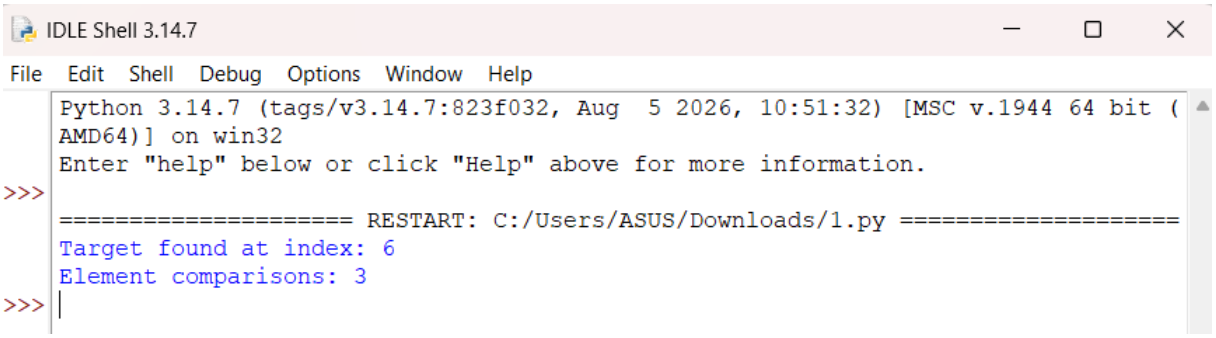
    return -1, comparisons

arr = [3, 7, 11, 15, 19, 23, 27, 31, 35]

target = 27
index, comparisons = binary_search(arr, target)
if index != -1:
    print("Target found at index:", index)
else:
    print("Target not found")

print("Element comparisons:", comparisons)
```

Output



9. Recurrence Relation

For a successful search, the algorithm may terminate at any midpoint. For an unsuccessful search, it repeatedly searches a remaining interval of approximately half the previous size.

A worst-case recurrence is

$$T(n) = T(n/2) + c$$

where c represents the constant amount of work needed to calculate the midpoint and compare the target.

Expanding the recurrence gives

$$T(n) = T(n/2) + c$$

$$T(n) = T(n/4) + 2c$$

$$T(n) = T(n/8) + 3c$$

After k steps, the remaining size is approximately

$$\frac{n}{2^k}$$

The search ends when this value becomes at most one:

$$\frac{n}{2^k} \leq 1$$

Therefore,

$$2^k \geq n$$

Taking logarithms,

$$k \geq \log_2 n$$

Hence,

$$\boxed{T(n) = O(\log n)}$$

Binary Search is logarithmic because the amount of remaining work is halved in each iteration.

10. Factors Affecting Performance

The number of comparisons depends on the target's location and the size of the array.

- A target found at the first midpoint requires very little work.
- A target found near the end of the search may require more iterations.
- An unsuccessful search may reach the point where the search interval becomes empty.

- An empty array returns NOT_FOUND immediately.
- Duplicate elements do not prevent the algorithm from finding a matching element, but the standard version does not guarantee the first or last occurrence.

If the task requires the first occurrence of a repeated value, the decision rule must be modified to continue searching the left side after finding a match.

11. Comparison With Linear Search

Feature	Linear Search	Binary Search
Input requirement	Can process unsorted data	Requires sorted data
Basic strategy	Examine elements sequentially	Eliminate half the interval
Worst-case time	O(n)	O(log n)
Additional space, iterative	O(1)	O(1)
Suitable use	Small or unsorted datasets	Large sorted datasets

Binary Search is especially useful when the dataset is already sorted and repeated searches are required. However, if the data is unsorted and only one search is needed, sorting the data first may cost more than simply performing Linear Search.

12. Conclusion

Binary Search demonstrates the power of elimination in algorithm design. By using the sorted order of the data, the algorithm determines which half of the search interval can no longer contain the target.
The method reduces the search space exponentially, giving a worst-case time complexity of O(log n). Its correctness depends on the ordering condition, its termination depends on the boundaries crossing, and its efficiency comes from discarding large portions of the input after each comparison.

Q2. THE QUALITY OF DECOMPOSITION IN SORTING

1. Introduction

Sorting is the process of arranging data in a specified order, usually ascending or descending. It is an important operation in databases, searching systems, data analytics, information retrieval, and scientific computing. Divide-and-Conquer sorting algorithms solve the sorting problem by breaking a large array into smaller subproblems. Merge Sort and Quick Sort are two important examples.Although both algorithms use recursion, they differ in the way they divide the input and combine the results. Merge Sort uses a predictable division into two approximately equal halves. Quick Sort divides the array according to a selected pivot, and the quality of the partition depends heavily on pivot selection. The purpose of this investigation is to compare their decomposition strategies, analyse their recursion structures, and understand how partition quality influences practical and theoretical performance.

2. Merge Sort

2.1 Principle

Merge Sort divides the input array into two approximately equal halves. It recursively sorts each half and then merges the two sorted halves into a single sorted array.

The algorithm follows three stages.

- Divide: Split the array at its midpoint.
- Conquer: Recursively sort the left and right subarrays.
- Combine: Merge the two sorted subarrays by repeatedly selecting the smallest remaining element.

Unlike Quick Sort, Merge Sort does not depend on a pivot. Its decomposition is determined by the midpoint of the current array.

2.2 Example

Consider the array

$$A = [38,27,43,3,9,82,10,5]$$

The first division produces

$$[38,27,43,3] \quad [9,82,10,5]$$

These halves are divided again:

$$[38,27] \quad [43,3] \quad [9,82] \quad [10,5]$$

The process continues until each subarray contains one element.

The singleton arrays are then merged in sorted order:

$$[27,38], \quad [3,43], \quad [9,82], \quad [5,10]$$

The next merge level produces

$$[3,27,38,43] \quad [5,9,10,82]$$

Finally, the two sorted halves are merged:

$$\boxed{[3,5,9,10,27,38,43,82]}$$

2.3 Pseudocode

```
ALGORITHM MergeSort(A, left, right)
INPUT:
   A - array
   left, right - current subarray boundaries
OUTPUT:
   A sorted in ascending order
BEGIN
   IF left ≥ right THEN
      RETURN
   END IF
```

```
    mid ← FLOOR((left + right) / 2)
    MergeSort(A, left, mid)
    MergeSort(A, mid + 1, right)
    Merge(A, left, mid, right)
END
```

The merge operation is defined as follows.

```
ALGORITHM Merge(A, left, mid, right)
INPUT:
    A - array
    left, mid, right - boundaries of two sorted subarrays
OUTPUT:
    The combined subarray sorted in ascending order
BEGIN
    Create temporary arrays L and R
    Copy A[left...mid] into L
    Copy A[mid+1...right] into R
    i ← 0
    j ← 0
    k ← left
    WHILE i < LENGTH(L) AND j < LENGTH(R) DO
        IF L[i] ≤ R[j] THEN
            A[k] ← L[i]
            i ← i + 1
        ELSE
            A[k] ← R[j]
            j ← j + 1
        END IF
        k ← k + 1
    END WHILE
    WHILE i < LENGTH(L) DO
        A[k] ← L[i]
        i ← i + 1
        k ← k + 1
    END WHILE
    WHILE j < LENGTH(R) DO
        A[k] ← R[j]
```

```
    j ← j + 1
    k ← k + 1
  END WHILE
END
```

2.4 Recurrence Relation

Merge Sort divides an array into two subproblems of size approximately \(n/2\). The merge operation processes all \(n\) elements and therefore requires linear time.

Its recurrence is

$$T(n) = 2T(n/2) + cn$$

where:

- $2T(n/2)$ represents the cost of sorting two halves.
- cn represents the merging cost.
- c is a constant associated with processing each element.

Using the Master Theorem,

$$a = 2, \qquad b = 2, \qquad f(n) = \Theta(n)$$

Since

$$n^{\log_b a} = n^{\log_2 2} = n$$

the recurrence falls into the case where the nonrecursive work is $\Theta(n)$.

Therefore,

$$\boxed{T(n) = \Theta(n \log n)}$$

The time complexity is $\Theta(n \log n)$ in the best, average, and worst cases.

2.5 Advantages

- Predictable decomposition into approximately equal halves.
- Guaranteed $\Theta(n \log n)$ time.
- Stable sorting when equal elements are merged consistently.
- Suitable for linked lists and external sorting.
- Reliable performance even when the input is already sorted or reverse sorted.

2.6 Limitations

- Array-based implementations generally require $\Theta(n)$ additional temporary storage.
- Merging introduces data movement.
- Recursive calls add overhead.
- For small arrays, simpler algorithms may perform better because their constant overhead is lower.

3. Quick Sort

3.1 Principle

Quick Sort selects an element called a pivot and partitions the array around it.

After partitioning, elements smaller than or equal to the pivot are placed on one side, while larger elements are placed on the other side, according to the chosen partition convention.

The pivot is placed in its final sorted position, and the two resulting subarrays are sorted recursively.

Quick Sort does not normally require a separate merge operation. Its major work occurs during partitioning.

3.2 Pivot Selection

Common pivot strategies include:

1. First element as pivot.
2. Last element as pivot.
3. Middle-position element as pivot.
4. Randomly selected pivot.
5. Median-of-three pivot, selected from the first, middle, and last elements.

No pivot strategy is universally best for every dataset. A pivot that creates balanced partitions on one input may create severely unbalanced partitions on another.

3.3 Balanced Partitioning

Suppose an array of eight elements is divided into partitions of sizes four and three, with the pivot occupying its final position.

The resulting recursive structure is approximately balanced. Each level processes the elements in the current subproblems, while the subproblem sizes decrease rapidly.

The recurrence for a balanced split is approximately

$$T(n) = 2T(n/2) + cn$$

Therefore,

$$\boxed{T(n) = \Theta(n \log n)}$$

This is the favourable behaviour of Quick Sort.

3.4 Severely Unbalanced Partitioning

Consider a partition that produces one empty subarray and another containing $(n - 1)$ elements.

The recurrence becomes

$$T(n) = T(n - 1) + cn$$

Expanding,

$$T(n) = T(n - 2) + c(n - 1) + cn$$

Continuing,

$$T(n) = c(n + (n - 1) + (n - 2) + \cdots + 1)$$

Using the arithmetic-series formula,

$$T(n) = c\frac{n(n+1)}{2}$$

Therefore,

$$\boxed{T(n) = \Theta(n^2)}$$

The algorithm repeatedly processes nearly the entire remaining array, resulting in quadratic time.

3.5 Python Implementation of Both Algorithms

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])

    return result

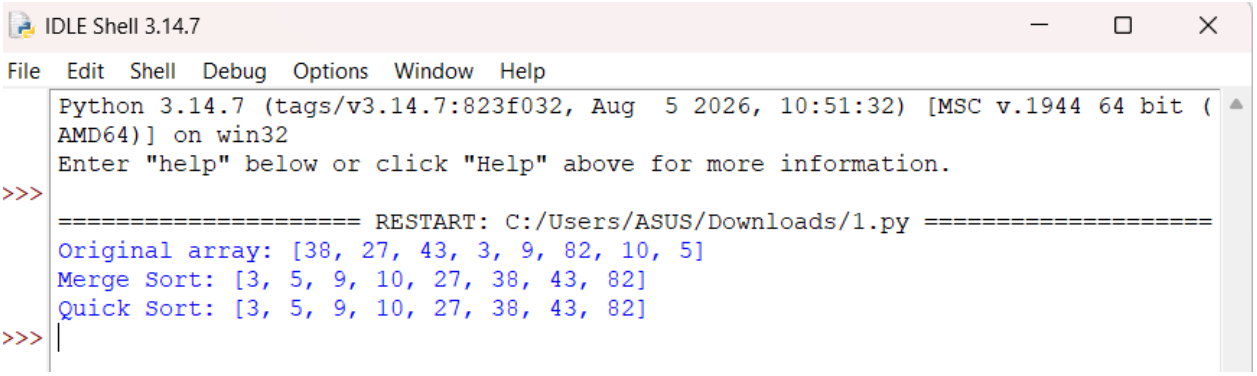
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[-1]
    left = []
    right = []
    for value in arr[:-1]:
        if value <= pivot:
            left.append(value)
        else:
            right.append(value)

    return quick_sort(left) + [pivot] + quick_sort(right)

data = [38, 27, 43, 3, 9, 82, 10, 5]

print("Original array:", data)
print("Merge Sort:", merge_sort(data))
print("Quick Sort:", quick_sort(data))
```

Output



3.6 Experimental Conditions

To investigate pivot quality, test the following types of input.

Balanced-partition test: Use randomly distributed values and record the partition sizes at each recursive call. Random input does not guarantee balanced partitions, so actual partition sizes must be recorded.

Sorted-input test: Use increasing-order data and a last-element pivot. This is a deliberately unfavourable combination because the pivot may repeatedly be the largest element.

Reverse-sorted test: Use decreasing-order data and record the resulting partition sizes.

Repeated-value test: Use an array containing many duplicate values. This helps reveal the effect of the partition convention and whether a three-way partition would be more appropriate.

The experiment should record:

- Input size.
- Pivot strategy.
- Size of each partition.
- Number of partition operations.
- Maximum recursion depth.
- Execution time over repeated runs.

The results must be collected from the actual implementation used for the assignment.

3.7 Comparison of Merge Sort and Quick Sort

Feature	**Merge Sort**	**Quick Sort**
Decomposition	Midpoint split	Pivot-based partition
Best-case time	O(n log n)	O(n log n)
Average-case time	O(n log n)	O(n log n)
Worst-case time	O(n log n)	O(n^2)
Auxiliary array storage	O(n)	Depends on implementation
Typical recursion depth	O(log n)	Average O(log n), worst O(n)
Stability	Stable when implemented appropriately	Usually unstable
Performance predictability	High	Depends on partition quality

For the simple list-based Quick Sort shown above, additional lists consume memory beyond the recursion stack. Its space requirements are therefore different from an in-place Quick Sort implementation.

3.8 Why Merge Sort Has More Predictable Decomposition

Merge Sort divides the input using its midpoint, regardless of the values or their original order. Each level therefore has a predictable structure, and the number of levels is logarithmic.

Quick Sort depends on the pivot's position relative to the other values. If the pivot is close to the median, the partitions are balanced. If it is close to an extreme, one partition may contain almost all the remaining elements.

The key difference is that Merge Sort controls subproblem sizes directly, whereas Quick Sort's partition sizes emerge from the input and pivot selection.

3.9 Conclusion

Merge Sort and Quick Sort both demonstrate Divide-and-Conquer design, but their performance depends on different properties.

Merge Sort's midpoint-based decomposition gives predictable O(n log n) performance. Quick Sort can achieve O(n log n) average-case behaviour and efficient in-place partitioning, but repeatedly unbalanced partitions can lead to O(n^2) time and linear recursion depth.

The experimental investigation should therefore evaluate partition quality, recursion depth, and execution time rather than relying only on the algorithm's name or average-case complexity.

Q3. BREAKING THE QUADRATIC BARRIER: KARATSUBA MULTIPLICATION

1. Introduction

Multiplication of large integers is a fundamental computational operation used in cryptography, scientific computing, numerical analysis, and arbitrary-precision arithmetic.

The traditional multiplication method is simple and effective for ordinary numbers. However, when the number of digits becomes very large, the number of elementary digit operations grows quadratically.

Karatsuba multiplication is a Divide-and-Conquer algorithm that reduces the number of recursive multiplications required to multiply two large integers. Instead of performing four smaller multiplications after splitting the operands, it performs only three.

This reduction changes the asymptotic time complexity from O(n^2) to approximately O(n^{1.585}), where n represents the number of digits.

2. Standard Integer Multiplication

Suppose two n-digit integers X and Y are multiplied using the traditional digit-by-digit method.

Each digit of X may need to be multiplied by every digit of Y. Consequently, the number of elementary digit multiplications is proportional to

$$n \times n = n^2$$

The recurrence can be represented as

$$T(n) = T(n-1) + O(n)$$

because each additional digit introduces a linear amount of work.

Expanding this recurrence produces a sum proportional to

$$1 + 2 + 3 + \cdots + n$$

Therefore,

$$\boxed{T(n) = O(n^2)}$$

3. Decomposition of Large Integers

Karatsuba multiplication divides each integer into high-order and low-order parts.

Suppose X and Y each contain approximately n digits. Choose a splitting position m, typically around half the number of digits.

Represent the integers as

$$X = aB^m + b$$
$$Y = cB^m + d$$

where:

- B is the numerical base, usually 10 for decimal representation.
- a and c are the high-order parts.
- b and d are the low-order parts.
- m is the number of digits assigned to each low-order part.

Multiplying the expressions gives

$$XY = (aB^m + b)(cB^m + d)$$

Expanding,

$$XY = acB^{2m} + (ad + bc)B^m + bd$$

A direct Divide-and-Conquer implementation would recursively compute four products:

$$ac, \quad ad, \quad bc, \quad bd$$

Karatsuba's key observation is that the two middle products can be obtained using only one additional multiplication.

4. Karatsuba's Algebraic Transformation

Define three products:

$$z_2 = ac$$
$$z_0 = bd$$
$$z_1 = (a + b)(c + d)$$

Expanding z_1 ,

$$z_1 = ac + ad + bc + bd$$

Therefore,

$$z_1 - z_2 - z_0 = ad + bc$$

The original product can now be reconstructed as

$$\boxed{XY = z_2B^{2m} + (z_1 - z_2 - z_0)B^m + z_0}$$

Only three recursive multiplications are required:

1. ac
2. bd
3. $(a + b)(c + d)$

Worked example

Calculate

$$1234 \times 5678$$

Split the numbers at two digits:

$$1234 = 12(100) + 34$$

$$5678 = 56(100) + 78$$

Therefore,

$$a = 12,\ b = 34,\ c = 56,\ d = 78,\ m = 2$$

Calculate the three products:

$$z_2 = 12 \times 56 = 672$$

$$z_0 = 34 \times 78 = 2652$$

$$z_1 = (12 + 34)(56 + 78)$$

$$z_1 = 46 \times 134 = 6164$$

The middle term is

$$z_1 - z_2 - z_0 = 6164 - 672 - 2652 = 2840$$

Reconstruct the answer:

$$XY = 672(10000) + 2840(100) + 2652$$

$$XY = 6{,}720{,}000 + 284{,}000 + 2{,}652$$

$$\boxed{1234 \times 5678 = 7{,}006{,}652}$$

The result matches the conventional multiplication result.

5. Language-Independent Pseudocode

```
ALGORITHM Karatsuba(X, Y)
INPUT:
   X, Y - non-negative integers
OUTPUT:
   Product X × Y
BEGIN
   IF X < 10 OR Y < 10 THEN
      RETURN X × Y
   END IF
   n ← MAX(NumberOfDigits(X), NumberOfDigits(Y))
   m ← FLOOR(n / 2)
   B ← 10^m
   a ← FLOOR(X / B)
   b ← X MOD B
   c ← FLOOR(Y / B)
```

d ← Y MOD B

z2 ← Karatsuba(a, c)

z0 ← Karatsuba(b, d)

z1 ← Karatsuba(a + b, c + d)

middle ← z1 - z2 - z0

RETURN z2 × B^(2m) + middle × B^m + z0

END

The pseudocode uses a small-input base case and recursively decomposes larger integers. In a practical implementation, the splitting position and base case must be handled carefully to avoid incorrect results when the operands have different lengths.

6. Python Implementation

```python
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y

    n = max(len(str(x)), len(str(y)))
    m = n // 2

    base = 10 ** m

    a = x // base
    b = x % base

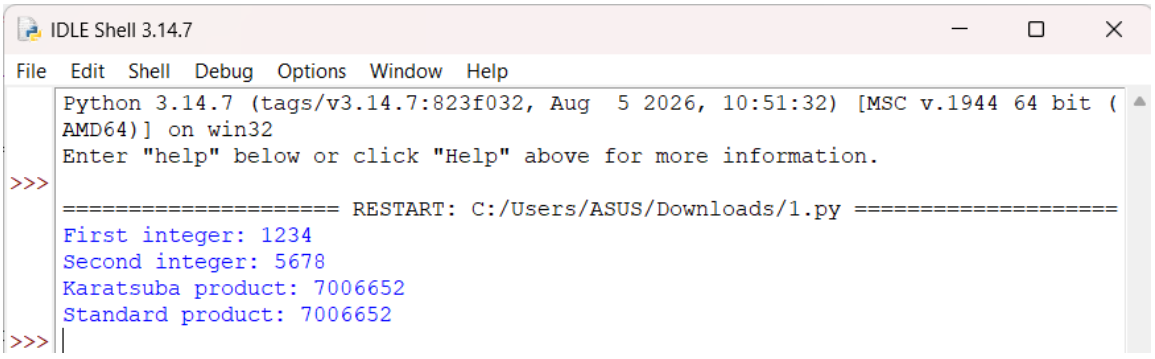
    c = y // base
    d = y % base
    z2 = karatsuba(a, c)
    z0 = karatsuba(b, d)
    z1 = karatsuba(a + b, c + d)
    middle = z1 - z2 - z0

    return (
        z2 * (base ** 2)
        + middle * base
        + z0
    )

x = 1234
y = 5678

print("First integer:", x)
print("Second integer:", y)
print("Karatsuba product:", karatsuba(x, y))
print("Standard product:", x * y)
```

Output



7. Recurrence Relation

Karatsuba divides each operand into halves and performs three recursive multiplications.

The recurrence is

$$T(n) = 3T(n/2) + O(n)$$

Here:

- $3T(n/2)$ represents the three recursive multiplications.
- $O(n)$ represents splitting, additions, subtractions, and reconstruction.

Applying the Master Theorem,

$$a = 3, \qquad b = 2, \qquad f(n) = O(n)$$

The critical exponent is

$$n^{\log_b a} = n^{\log_2 3}$$

Since

$$\log_2 3 \approx 1.585$$

and the nonrecursive work is asymptotically smaller $n^{1.585}$, the recurrence gives

$$\boxed{T(n) = \Theta\left(n^{\log_2 3}\right)}$$

or approximately

$$\boxed{T(n) = \Theta(n^{1.585})}$$

This is asymptotically faster than the conventional $O(n^2)$ multiplication method.

8. Why Three Multiplications Matter

The traditional recursive method performs four multiplications of half-sized operands:

$$T(n) = 4T(n/2) + O(n)$$

Its critical exponent is

$$\log_2 4 = 2$$

Therefore, its recurrence produces quadratic complexity.

Karatsuba reduces the number of recursive multiplications from four to three. Even though it performs extra additions and subtractions, the reduction in recursive multiplication count improves the asymptotic growth rate.

The improvement becomes increasingly important as the operands grow.

9. Crossover Experiment

Asymptotic complexity does not directly determine performance for small inputs. Karatsuba introduces overhead through:

- Recursive function calls.
- Converting or processing large integers.
- Splitting operands.

- Computing three intermediate products.
- Adding and subtracting intermediate values.
- Reconstructing the final result.

For small integers, these extra operations may cost more than simply multiplying the operands directly.

To identify the crossover point, compare conventional multiplication and Karatsuba multiplication using progressively larger operands.

The experiment should include digit lengths such as 8, 16, 32, 64, 128, 256, 512, and larger sizes supported by the implementation.

For each size:

1. Generate several pairs of integers with the required digit length.
2. Verify that both methods produce identical products.
3. Repeat the timing experiment multiple times.
4. Calculate a representative time, such as the median.
5. Record recursive multiplication counts.
6. Identify the approximate range where Karatsuba becomes faster.

The crossover point depends on the implementation, language, hardware, base case, and measurement method. It should not be claimed as a universal constant.

10. Experimental Analysis

The results should be interpreted in terms of both asymptotic complexity and constant overhead.

At smaller input sizes, direct multiplication is expected to be competitive because it has little overhead. As the number of digits increases, Karatsuba's lower asymptotic exponent can become advantageous.

However, the actual crossover point must be measured. An implementation that repeatedly converts integers to strings may incur additional costs that are not inherent to Karatsuba's mathematical recurrence.

To make the experiment more reliable:

- Use the same operand pairs for both methods.
- Repeat each measurement.
- Separate correctness testing from performance testing.
- Avoid including random-number generation inside the timed region.
- Use the same Python runtime and machine.
- Record the digit lengths, not just the numerical magnitudes.

Conclusion

Karatsuba multiplication demonstrates how algebraic reformulation can reduce the number of recursive subproblems. It replaces four half-sized multiplications with three, changing the time complexity from quadratic to approximately $O(n^{1.585})$.

Nevertheless, the algorithm's practical benefit depends on operand size and implementation overhead.

This investigation shows why both mathematical analysis and experimental validation are required when selecting an algorithm.

Q4. THE REALITY OF STRASSEN'S MATRIX MULTIPLICATION

1. Introduction

Matrix multiplication is widely used in scientific computing, machine learning, computer graphics, physics simulations, and numerical analysis.

For two square matrices of order n, conventional matrix multiplication computes each output element using n multiplications and approximately n-1 additions. Since there are n^2 output elements, the total arithmetic work is proportional to n^3.

Strassen's algorithm uses Divide and Conquer to reduce the number of recursive block multiplications from eight to seven. This changes the asymptotic time complexity from $O(n^3)$ to approximately $O(n^{2.807})$.

However, Strassen's algorithm also introduces additional additions, subtractions, recursive calls, temporary matrices, and possible padding. Its superior asymptotic complexity therefore does not guarantee better performance for every matrix size.

2. Conventional Matrix Multiplication

Let \(A\) and \(B\) be two $n \times n$ matrices. Their product $C = AB$ is defined by

$$C_{ij} = \sum_{k=1}^{n} A_{ik}\, B_{kj}$$

Each output element is obtained by multiplying corresponding row and column elements and summing the results.

For example,

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Then

$$C_{11} = 1(5) + 2(7) = 19$$
$$C_{12} = 1(6) + 2(8) = 22$$
$$C_{21} = 3(5) + 4(7) = 43$$
$$C_{22} = 3(6) + 4(8) = 50$$

Therefore,

$$C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

3. Conventional Algorithm

ALGORITHM MatrixMultiply(A, B, n)

```
INPUT:
   A, B - two n × n matrices
OUTPUT:
   Product matrix C = A × B
BEGIN
   Create matrix C filled with zeros
   FOR i ← 0 TO n - 1 DO
      FOR j ← 0 TO n - 1 DO
         FOR k ← 0 TO n - 1 DO
            C[i][j] ← C[i][j] + A[i][k] × B[k][j]
         END FOR
      END FOR
   END FOR
   RETURN C
END
```

The three nested loops each execute a number of iterations proportional to \(n\).

Therefore,

$$\boxed{T(n) = \Theta(n^3)}$$

Conventional multiplication is straightforward, predictable, and often highly optimized by numerical libraries.

4. Block Matrix Decomposition

For simplicity, consider square matrices whose dimensions are even. Divide each matrix into four equal-sized blocks:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

$$B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Each block has dimensions $n/2 \times n/2$.

The product is

$$C = AB$$

with blocks

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

Computing these four expressions directly requires eight block multiplications.

If each block multiplication is performed recursively, the recurrence becomes

$$T(n) = 8T(n/2) + O(n^2)$$

The resulting time complexity is

$$\boxed{T(n) = \Theta(n^3)}$$

Thus, simply dividing the matrices into blocks does not improve the asymptotic complexity. Strassen's improvement comes from reducing the number of recursive block multiplications.

5. Strassen's Seven Products

Strassen defines seven intermediate matrices:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$
$$M_2 = (A_{21} + A_{22})B_{11}$$
$$M_3 = A_{11}(B_{12} - B_{22})$$
$$M_4 = A_{22}(B_{21} - B_{11})$$
$$M_5 = (A_{11} + A_{12})B_{22}$$
$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$
$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

These expressions require only seven block multiplications.

The product blocks are reconstructed using

$$C_{11} = M_1 + M_4 - M_5 + M_7$$
$$C_{12} = M_3 + M_5$$
$$C_{21} = M_2 + M_4$$
$$C_{22} = M_1 - M_2 + M_3 + M_6$$

The transformation replaces one block multiplication with additional matrix additions and subtractions.

6. Why the Transformation Is Correct

The correctness of Strassen's algorithm follows from algebraic expansion.

For the first output block,

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

Substitute the definitions:

$$\begin{aligned} C_{11} = {} & (A_{11} + A_{22})(B_{11} + B_{22}) \\ & + A_{22}(B_{21} - B_{11}) \\ & - (A_{11} + A_{12})B_{22} \\ & + (A_{12} - A_{22})(B_{21} + B_{22}) \end{aligned}$$

Expanding the products and cancelling equal positive and negative terms leaves

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

which is exactly the conventional formula.

The other three output blocks can be verified by expanding their corresponding expressions in the same manner. Each reconstruction cancels the additional intermediate terms and produces the conventional block-

matrix multiplication result.

This establishes that the algorithm changes the computational method without changing the mathematical product.

7. Recurrence Relation

Strassen performs seven recursive multiplications on matrices of size $(n/2)$. The matrix additions and subtractions require $O(n^2)$ work.

Therefore,

$$T(n) = 7T(n/2) + O(n^2)$$

Applying the Master Theorem,

$$a = 7, \qquad b = 2, \qquad f(n) = O(n^2)$$

The critical exponent is

$$\log_2 7 \approx 2.807$$

Since n^2 grows more slowly than $n^{\log_2 7}$, the recurrence gives

$$\boxed{T(n) = \Theta\left(n^{\log_2 7}\right)}$$

or approximately

$$\boxed{T(n) = \Theta(n^{2.807})}$$

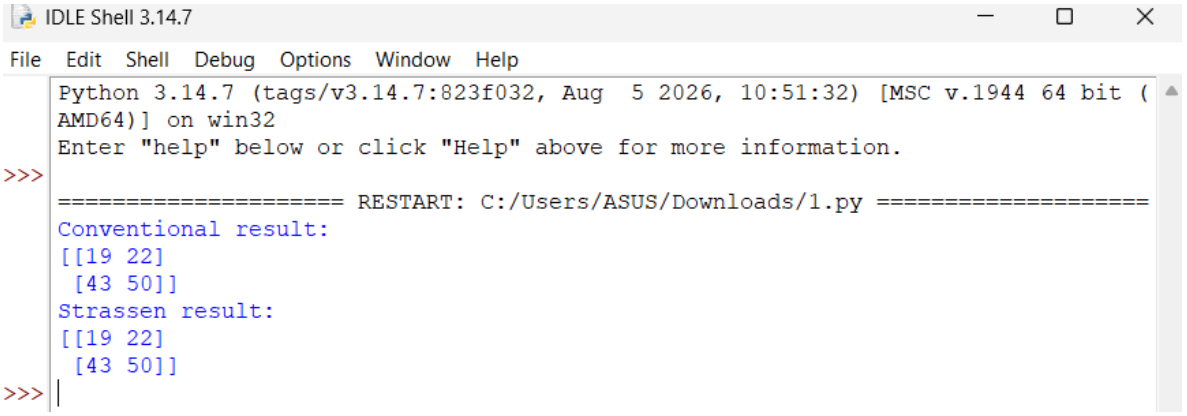
This is asymptotically better than conventional matrix multiplication's $\Theta(n^3)$ complexity.

8. Python Implementation

```python
import numpy as np
def conventional_multiply(A, B):
    return A @ B
def strassen(A, B, threshold=32):
    n = A.shape[0]
    if n <= threshold:
        return A @ B
    mid = n // 2
    A11 = A[:mid, :mid]
    A12 = A[:mid, mid:]
    A21 = A[mid:, :mid]
    A22 = A[mid:, mid:]
    B11 = B[:mid, :mid]
    B12 = B[:mid, mid:]
    B21 = B[mid:, :mid]
    B22 = B[mid:, mid:]
    M1 = strassen(A11 + A22, B11 + B22, threshold)
    M2 = strassen(A21 + A22, B11, threshold)
    M3 = strassen(A11, B12 - B22, threshold)
    M4 = strassen(A22, B21 - B11, threshold)
    M5 = strassen(A11 + A12, B22, threshold)
    M6 = strassen(A21 - A11, B11 + B12, threshold)
    M7 = strassen(A12 - A22, B21 + B22, threshold)
    C11 = M1 + M4 - M5 + M7
    C12 = M3 + M5
    C21 = M2 + M4
    C22 = M1 - M2 + M3 + M6
    top = np.hstack((C11, C12))
    bottom = np.hstack((C21, C22))
    return np.vstack((top, bottom))
def strassen_general(A, B, threshold=32):
    A = np.asarray(A)
    B = np.asarray(B)
    if A.shape[1] != B.shape[0]:
        raise ValueError("Incompatible matrix dimensions")
    rows, inner = A.shape
    _, cols = B.shape

    size = max(rows, inner, cols)
    padded_size = 1
    while padded_size < size:
        padded_size *= 2
    dtype = np.result_type(A.dtype, B.dtype, np.int64)
    A_pad = np.zeros((padded_size, padded_size), dtype=dtype)
    B_pad = np.zeros((padded_size, padded_size), dtype=dtype)
    A_pad[:rows, :inner] = A
    B_pad[:inner, :cols] = B
```

Output



9. Padding Non-Power-of-Two Dimensions

Strassen's recursive implementation is simplest when the matrix dimension is a power of two.

If the matrix dimension is not a power of two, the matrices can be padded with zeros until their dimensions reach the next power of two.

For example, a 5×5 matrix can be extended to an 8×8 matrix.

The original matrix is placed in the upper-left region, and the remaining cells are filled with zeros. Both padded matrices are multiplied, after which the extra rows and columns are removed.

Because multiplying by zero does not change the original product, the upper-left region of the resulting matrix contains the required answer.

However, padding introduces extra work. A matrix with dimension n may be expanded to a dimension close to $2n$, depending on the original size. The algorithm then processes the larger padded matrices, increasing computation and memory usage.

For rectangular matrices, padding to a square matrix can be particularly wasteful. More specialized implementations can reduce this overhead.

10. Experimental Design

The experiment should compare conventional multiplication and Strassen multiplication using identical matrices.

Suggested dimensions include:

$$16,\ 32,\ 64,\ 128,\ 256,\ 512$$

The experiment should:

1. Generate matrices with reproducible random values.
2. Compute the product using both methods.
3. Check that the results agree.
4. Repeat each timing measurement.
5. Record multiplication counts or theoretical recursive multiplication counts.
6. Record the execution times and memory-related observations.

7. Repeat the experiment with a non-power-of-two dimension.

The implementation's base-case threshold must be documented because it influences the observed crossover point.

11. Engineering Analysis

Strassen's algorithm may perform worse for smaller matrices because:

- It introduces many matrix additions and subtractions.
- Each recursive call requires intermediate arrays.
- Temporary allocations increase memory pressure.
- Padding may increase the effective matrix dimension.
- Recursive control overhead is significant for small subproblems.
- Memory locality and cache behaviour may favour a highly optimized conventional implementation.

For large matrices, the lower asymptotic exponent may eventually outweigh the extra overhead. The crossover range depends on hardware, numeric representation, memory management, matrix layout, and implementation quality.

12. Joint Engineering Decision

Conventional multiplication should be considered when the matrices are small, memory is limited, the implementation is highly optimized, or the overhead of recursive decomposition exceeds its computational savings.

It is also suitable when dimensions are irregular and padding would cause significant waste, or when numerical accuracy and reproducibility requirements make the additional arithmetic transformations undesirable.

Strassen is more attractive when matrices are sufficiently large, the implementation handles temporary storage efficiently, and experiments demonstrate that its lower asymptotic growth translates into an actual performance improvement.

13. Conclusion

Strassen's algorithm reduces recursive matrix multiplications from eight to seven and achieves a theoretical time complexity of approximately $O(n^{2.807})$. Its algebraic reconstruction produces the same result as conventional block multiplication.

Nevertheless, algorithm selection must account for constant factors, memory usage, padding, recursion, and numerical considerations. The experiment is necessary to identify the range in which Strassen becomes practically advantageous for the chosen implementation.

Q5. UPGRADING CLOSEST PAIR USING DIVIDE AND CONQUER

1. Introduction

The Closest Pair of Points problem is a classical computational geometry problem. Given a collection of points

in a two-dimensional plane, the objective is to identify the pair whose Euclidean distance is the smallest.
This problem has applications in geographical information systems, collision detection, clustering, computer graphics, spatial databases, and sensor-network analysis.
The brute-force approach compares every pair of points. Although this method is easy to implement, the number of comparisons grows quadratically with the number of points.
A Divide-and-Conquer approach improves the asymptotic time complexity to \(O(n\log n)\) by sorting the points, solving smaller subproblems, and carefully examining points near the dividing boundary.

2. Problem Definition

Given a set of points

$$P = \{p_1, p_2, \dots, p_n\}$$

where each point is represented by coordinates (x_i, y_i) , find the pair of distinct points having the smallest Euclidean distance.

For two points

$$p_i = (x_i, y_i)$$

and

$$p_j = (x_j, y_j)$$

their Euclidean distance is

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

The algorithm must return the closest pair and its distance.

3. Brute-Force Approach

The brute-force method computes the distance between every possible pair of points and keeps track of the smallest distance.

There are

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

distinct pairs.

Therefore, the total number of pairwise distance calculations is proportional to \(n^2\).

The time complexity is

$$\boxed{T(n) = \Theta(n^2)}$$

The brute-force algorithm is suitable for small datasets and provides a useful baseline for evaluating the Divide-and-Conquer method.

Python implementation

```python
import math

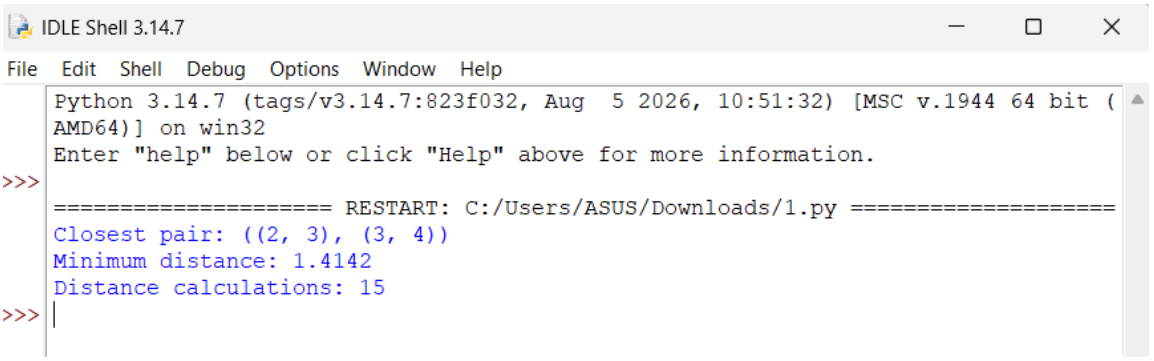
def closest_pair_brute_force(points):
    n = len(points)
    if n < 2:
        raise ValueError("At least two points are required")
    best_distance = float("inf")
    best_pair = None
    comparisons = 0
    for i in range(n):
        for j in range(i + 1, n):
            x1, y1 = points[i]
            x2, y2 = points[j]
            distance = math.sqrt(
                (x1 - x2) ** 2 + (y1 - y2) ** 2
            )
            comparisons += 1
            if distance < best_distance:
                best_distance = distance
                best_pair = (points[i], points[j])

    return best_pair, best_distance, comparisons

points = [
    (2, 3),
    (12, 30),
    (40, 50),
    (5, 1),
    (12, 10),
    (3, 4)
]
pair, distance, comparisons = closest_pair_brute_force(points)

print("Closest pair:", pair)
print("Minimum distance:", round(distance, 4))
print("Distance calculations:", comparisons)
```

Output



4. Divide-and-Conquer Design

The Divide-and-Conquer solution begins by sorting the points according to their \(x\)-coordinates.

It then divides the points into two halves:

- Left half P_L.
- Right half P_R.

The algorithm recursively finds the closest pair in each half.

Let the minimum distances returned by the two recursive calls be

$$d_L$$

and

$$d_R$$

The best distance obtained so far is

$$\boxed{\delta = \min(d_L, d_R)}$$

However, the closest pair might consist of one point from the left half and another from the right half. Such a pair would not be examined by either recursive call.
Therefore, the algorithm must examine points near the dividing boundary.

5. Why the Boundary Requires Special Treatment

Suppose the closest pair contains a point on each side of the dividing line. The horizontal separation between these points cannot exceed the best distance δ already obtained from the two halves.
If the horizontal separation were greater than δ, the Euclidean distance would necessarily be greater than δ, because the Euclidean distance is at least as large as the absolute horizontal separation.
Therefore, only points whose horizontal distance from the dividing line is less than δ need to be considered.
These points form a vertical strip.
The algorithm sorts or maintains the strip points by their y-coordinates and examines only a limited number of nearby points for each point in the strip.

6. Why the Strip Does Not Require All-Pairs Comparison

The boundary stage is efficient because the recursive solutions already establish that no two points within the same half are closer than δ.
This condition restricts how many points can be densely packed into a small region. Geometric packing arguments show that only a constant number of following points in the y-sorted strip need to be examined for each point in the standard two-dimensional algorithm.
Consequently, the strip-processing stage takes linear time rather than quadratic time.
The constant depends on the precise geometric argument and implementation, but the important complexity result is that each strip point requires only O(1) distance comparisons.
Therefore, the combine step is O(n), provided the points are organised by y-coordinate efficiently.

7. Recursive Structure

The recurrence is

$$T(n) = 2T(n/2) + O(n)$$

where:

- $2T(n/2)$ represents the two recursive subproblems.
- $O(n)$ represents merging or maintaining the \(y\)-sorted order and processing the strip.

Applying the Master Theorem gives

$$\boxed{T(n) = \Theta(n \log n)}$$

This is an improvement over the brute-force algorithm's $\Theta(n^2)$ time complexity.
A critical implementation detail is that the y-sorted lists should be maintained or merged efficiently. Sorting the strip from scratch at every recursive level can introduce an extra logarithmic factor and weaken the intended time bound.

8. Pseudocode

```
ALGORITHM ClosestPair(P)
INPUT:
   P - set of at least two two-dimensional points
OUTPUT:
   Closest pair and their minimum Euclidean distance
BEGIN
   Sort P by x-coordinate
   Sort P by y-coordinate
   RETURN ClosestPairRecursive(Px, Py)
END
```

The recursive procedure is conceptually represented as follows.

```
ALGORITHM ClosestPairRecursive(Px, Py)
INPUT:
   Px - points ordered by x-coordinate
   Py - same points ordered by y-coordinate
OUTPUT:
   Closest pair and minimum distance
BEGIN
   n ← LENGTH(Px)
   IF n ≤ 3 THEN
      RETURN BruteForceClosestPair(Px)
   END IF
   Divide Px into left and right halves
   Partition Py into PyLeft and PyRight
   while preserving y-coordinate order
   (pairLeft, distanceLeft) ←
      ClosestPairRecursive(PxLeft, PyLeft)
   (pairRight, distanceRight) ←
      ClosestPairRecursive(PxRight, PyRight)
   IF distanceLeft ≤ distanceRight THEN
      bestPair ← pairLeft
      delta ← distanceLeft
   ELSE
      bestPair ← pairRight
      delta ← distanceRight
   END IF
```

Construct strip containing points whose
horizontal distance from the dividing line
is less than delta
Examine only the permitted following points
in y-coordinate order
Update bestPair and delta when a closer
pair is found
RETURN (bestPair, delta)

END

The pseudocode shows the essential structure. An implementation must define how points are assigned to the halves, how equal coordinates are handled, and how the strip comparison limit is enforced.

10. Correctness Argument

The correctness of the algorithm can be established by considering three possibilities for the globally closest pair.

Case 1: Both points are in the left half.

The recursive call on the left half finds the closest pair within that half. Therefore, the algorithm considers this possibility.

Case 2: Both points are in the right half.

The recursive call on the right half finds the closest pair within that half. Therefore, this possibility is also considered.

Case 3: The points belong to different halves.

If the points form the closest pair across the division, their horizontal separation is smaller than the best distance found within either half. Consequently, both points lie within the strip.

The strip-processing stage examines the necessary nearby points in \(y\)-coordinate order. Therefore, the cross-boundary closest pair is not omitted.

Since the algorithm examines the closest pair from both recursive halves and all necessary cross-boundary candidates, it returns a globally closest pair.

11. Experimental Comparison

The brute-force and Divide-and-Conquer methods must be tested on equivalent datasets.

Suggested input sizes include

$$16,\ 32,\ 64,\ 128,\ 256,\ 512,\ 1024$$

For each size, generate the same point set and run both algorithms.

Record:

- Number of points.
- Number of distance calculations.

- Execution time.
- Closest-pair distance.
- Whether both algorithms return the same minimum distance.

Distance calculations provide a useful measure of algorithmic work. Execution time provides a practical measure but is more sensitive to the implementation and system environment.

12. Conclusion

The Closest Pair problem demonstrates how geometric properties can make Divide and Conquer more efficient than brute-force enumeration.

The brute-force algorithm requires $\Theta(n^2)$ distance calculations. The Divide-and-Conquer method achieves $\Theta(n \log n)$ time by solving the left and right halves and restricting the boundary search to a narrow strip.

The strip does not require comparing every point with every other point because the minimum distance already established within the two halves imposes geometric restrictions on how many nearby candidates need to be checked.

Experimental validation should establish both correctness and reduced computational work.

Q6. ADVANCED DIVIDE-AND-CONQUER MINI-INVESTIGATIONS

Part A: Median of Medians for the k-th Smallest Element

1. Introduction

The selection problem finds the k-th smallest element in an unsorted array. QuickSelect solves this problem by choosing a pivot and partitioning the array, but its worst-case time complexity is $O(n^2)$.

The Median of Medians algorithm improves the worst-case performance by selecting a suitable pivot deterministically.

2. Working Principle

1. Divide the array into groups of five elements.
2. Find the median of each group.
3. Recursively find the median of these medians.
4. Use it as the pivot to partition the array.
5. Continue searching in the partition containing the k-th smallest element.

This strategy ensures that a sufficient number of elements can be eliminated during each partition.

3. Pseudocode

ALGORITHM MedianOfMedians(A, k)

BEGIN

Divide A into groups of five

Find the median of each group

Find the median of these medians

Partition A around the selected pivot

IF pivot is the k-th smallest element

RETURN pivot

ELSE IF k lies in the lower partition

Recursively search the lower partition

ELSE

Adjust k and search the upper partition

END IF

END

4. Python Code

```
def median_of_medians(arr, k):
    if not 1 <= k <= len(arr):
        raise ValueError("Invalid k")

    def select(a, k):
        if len(a) <= 5:
            return sorted(a)[k - 1]

        groups = [a[i:i + 5] for i in range(0, len(a), 5)]
        medians = [sorted(g)[len(g) // 2] for g in groups]

        pivot = select(medians, (len(medians) + 1) // 2)

        low = [x for x in a if x < pivot]
        equal = [x for x in a if x == pivot]
        high = [x for x in a if x > pivot]

        if k <= len(low):
            return select(low, k)

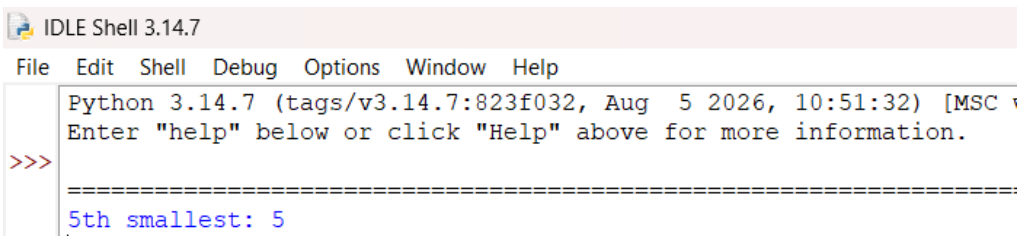
        if k <= len(low) + len(equal):
            return pivot

        return select(high, k - len(low) - len(equal))

    return select(arr, k)


arr = [9, 2, 7, 4, 1, 6, 3, 8, 5]
print("5th smallest:", median of medians(arr, 5))
```

Output



5. Complexity Analysis

The recurrence is

$$T(n) \leq T(n/5) + T(7n/10) + O(n)$$

Therefore, the worst-case time complexity is $O(n)$.

Algorithm	Expected Time	Worst-Case Time
QuickSelect	$O(n)$	$O(n^2)$
Median of Medians	$O(n)$	$O(n)$

6. Conclusion

Median of Medians guarantees worst-case linear time by selecting a balanced pivot. Although it introduces additional overhead, it is useful when predictable performance is important.

Part B: Meet-in-the-Middle for Subset Sum

1. Introduction

The Subset-Sum problem determines whether a subset of an integer array adds up to a given target.

For example, in the array $[3,7,2,5]$, a subset with target sum 10 is $[3,7]$

Exhaustive search examines all 2^n subsets. Meet-in-the-Middle reduces the search by dividing the array into two halves.

2. Working Principle

1. Divide the array into left and right halves.
2. Generate all subset sums for each half.
3. Store the right-half sums in a hash table.
4. For each left-half sum s, search for the complementary sum target-s.
5. If a match exists, combine the corresponding subsets.

3. Pseudocode

```
ALGORITHM MeetInTheMiddle(A, target)
BEGIN
  Divide A into Left and Right
  Generate all subset sums of Left
  Generate all subset sums of Right
  Store right-half sums
  FOR each left sum s
    required ← target - s
    IF required exists in right sums
      RETURN matching subset
    END IF
```

END FOR

RETURN NOT_FOUND

END

4. Python Code

```
def subset_sums(arr):
    sums = [(0, [])]

    for x in arr:
        sums += [(total + x, subset + [x])
                 for total, subset in sums]
    return sums

def meet_in_middle(arr, target):
    mid = len(arr) // 2
    left_sums = subset_sums(arr[:mid])
    right_sums = subset_sums(arr[mid:])

    right_lookup = {}
    for total, subset in right_sums:
        if total not in right_lookup:
            right_lookup[total] = subset

    for total, subset in left_sums:
        required = target - total

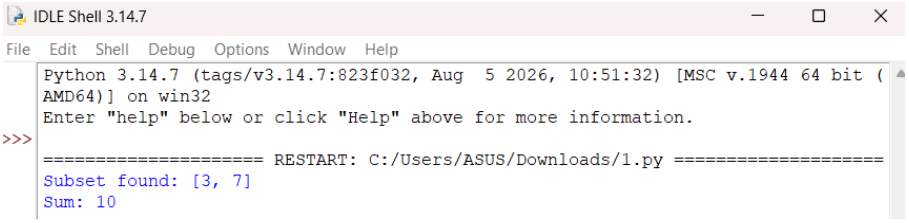
        if required in right_lookup:
            return subset + right_lookup[required]

    return None

arr = [3, 7, 2, 5, 8, 1]
target = 10

result = meet_in_middle(arr, target)
print("Subset found:", result)
print("Sum:", sum(result) if result is not None else "Not found")
```

Output



5. Complexity Analysis

Each half contains approximately $n/2$ elements, producing approximately $2^{n/2}$ subset sums.

- Time complexity: $O(2^{n/2})$ expected.
- Space complexity: $O(2^{n/2})$

Method	Time Complexity
Exhaustive Search	$O(2^n)$
Meet-in-the-Middle	$O(2^{n/2})$ expected

6. Conclusion

Meet-in-the-Middle reduces the search time by dividing the problem into two smaller parts. It is useful for moderate-sized inputs but requires additional memory to store subset sums.

Q7. Design Divide and Conquer from First Principles

1. Introduction

Divide and Conquer solves a problem by dividing it into smaller subproblems, solving them recursively, and combining their results. This question demonstrates the design process using the Maximum Subarray Sum problem.

Given an array of integers, the objective is to find the maximum sum of a non-empty contiguous subarray.

For example:

$$A = [-2,1,-3,4,-1,2,1,-5,4]$$

The maximum-sum subarray is $[4,-1,2,1]$, whose sum is 6.

2. Independent Design Proposals

Student A's proposal: Divide the array into two halves and find the maximum subarray sum in each half. However, this approach may miss a subarray crossing the midpoint.

Student B's proposal: Divide the array into two halves and consider three possibilities: the maximum subarray lies entirely in the left half, entirely in the right half, or crosses the midpoint.

The second proposal is more complete because it considers every possible location of the maximum subarray.

3. Final Joint Design

The final algorithm uses the following steps:

1. Divide the array at its midpoint.
2. Recursively find the maximum subarray sum in the left half.
3. Recursively find the maximum subarray sum in the right half.
4. Find the maximum suffix sum of the left half.
5. Find the maximum prefix sum of the right half.
6. Add these values to obtain the maximum crossing sum.
7. Return the maximum of the three results.

The recurrence is

$$T(n) = 2T(n/2) + O(n)$$

Using the Master Theorem, the time complexity is

$$\boxed{O(n \log n)}$$

4. Pseudocode

```
ALGORITHM MaximumSubarray(A, left, right)
BEGIN
  IF left = right
    RETURN A[left]
```

END IF

mid ← (left + right) / 2

leftMax ← MaximumSubarray(A, left, mid)

rightMax ← MaximumSubarray(A, mid + 1, right)

Find maximum suffix sum of left half

Find maximum prefix sum of right half

crossingMax ← suffix sum + prefix sum

RETURN MAX(leftMax, rightMax, crossingMax)

END

5. Python Code

```python
def maximum_subarray_sum(arr):
    if not arr:
        raise ValueError("Array must not be empty")

    def solve(left, right):
        if left == right:
            return arr[left]

        mid = (left + right) // 2

        left_max = solve(left, mid)
        right_max = solve(mid + 1, right)

        total = 0
        left_suffix = float("-inf")
        for i in range(mid, left - 1, -1):
            total += arr[i]
            left_suffix = max(left_suffix, total)

        total = 0
        right_prefix = float("-inf")

        for i in range(mid + 1, right + 1):
            total += arr[i]
            right_prefix = max(right_prefix, total)

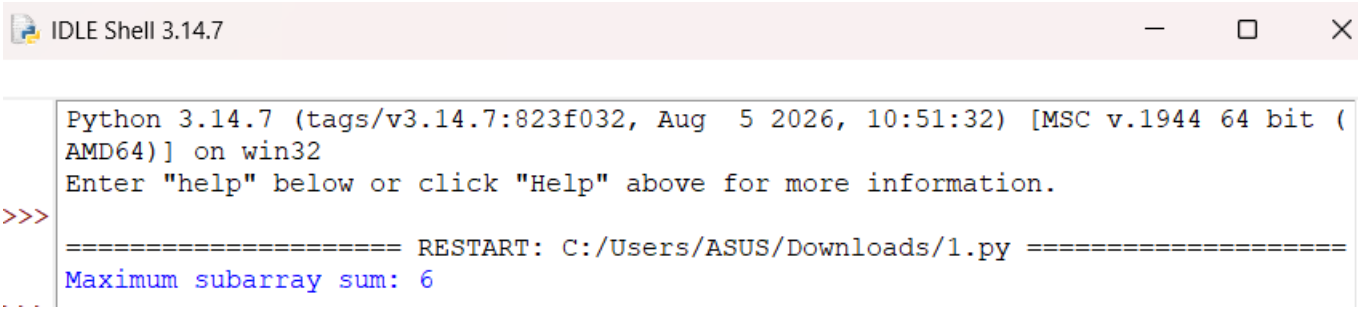
        crossing = left_suffix + right_prefix

        return max(left_max, right_max, crossing)

    return solve(0, len(arr) - 1)


arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
print("Maximum subarray sum:", maximum_subarray_sum(arr))
```

Output



6. Experimental Validation

The algorithm should be tested with different inputs to verify correctness.

Input	Expected Maximum Sum
[-2,1,-3,4,-1,2,1,-5,4]	6
[1,2,3,4]	10
[-8,-3,-6,-2,-5]	-2
[7]	7
[5,-2,3,-1,2]	7

A brute-force algorithm can be used as a baseline. It checks every contiguous subarray and has $O(n^2)$ time complexity. Comparing both methods on identical inputs verifies their results and demonstrates how computational work changes as the input size increases.

7. Joint Evaluation

The final design is correct because every maximum subarray must belong to one of three categories: the left half, the right half, or a subarray crossing the midpoint.

The algorithm reduces computational work by solving smaller subproblems and examining only the necessary crossing candidates. However, recursion introduces function-call overhead and requires additional stack space.

Divide and Conquer may be less suitable when subproblems overlap heavily, when an efficient iterative method already exists, or when recursion creates excessive overhead. For the maximum subarray problem, Kadane's algorithm achieves $O(n)$ time, making it more efficient for practical use.

8. Conclusion

This investigation demonstrates the process of designing a Divide-and-Conquer algorithm from first principles. The final solution includes a clear decomposition, base case, combination step, pseudocode, implementation, recurrence, and experimental validation.

The algorithm achieves $O(n \log n)$ time compared with the brute-force method's $O(n^2)$. It demonstrates that successful algorithm design requires both a suitable decomposition and a correct, efficient method for combining the subproblem results.